

# MediQuery: AI-Powered Data Analytics Platform

---

Platform Overview, Architecture, and Technical Specifications

## 1. Executive Summary

MediQuery is an advanced, AI-assisted data analytics and visualization platform designed to accelerate dashboard creation and data discovery. Unlike traditional platforms where end-users prompt AI constantly, MediQuery uses AI strategically during the initial data onboarding to generate a baseline dashboard. From there, Administrators take full control—manually building custom charts, defining complex metrics, and designing highly polished dashboards for end-users to consume and interact with. This approach guarantees extreme scalability, zero ongoing AI API costs for dashboard viewing, and absolute data privacy.

## 2. Key Features

- **AI-Assisted Onboarding:** Upon dataset upload, the AI analyzes the dataset's schema and instantly generates a foundational dashboard with baseline charts, saving Administrators hours of initial setup time.
- **Administrator-Driven Dashboard Design:** End-users do not prompt the AI to build charts. Instead, Administrators manually construct custom charts, KPIs, and layouts using a powerful drag-and-drop UI. This ensures maximum quality control and reliability.
- **Unlimited Interactive Visualizations:** Because Administrators build widgets manually through the UI, there is absolutely no limit to the number of charts that can be added to a dashboard. This bypasses all LLM token limits and API constraints.
- **Calculated Fields & Custom Metrics:** Administrators can define custom mathematical logic which the system accurately integrates into visual charts and filters.
- **Real-time Global Parameters:** Intuitive date pickers, dropdowns, and text inputs allow end-users to filter the Administrator-designed dashboard across all charts instantly.
- **Export & Sharing:** End-users can export the curated visualizations as high-quality PDFs or raw data as CSVs.

### 3. Data Connectivity

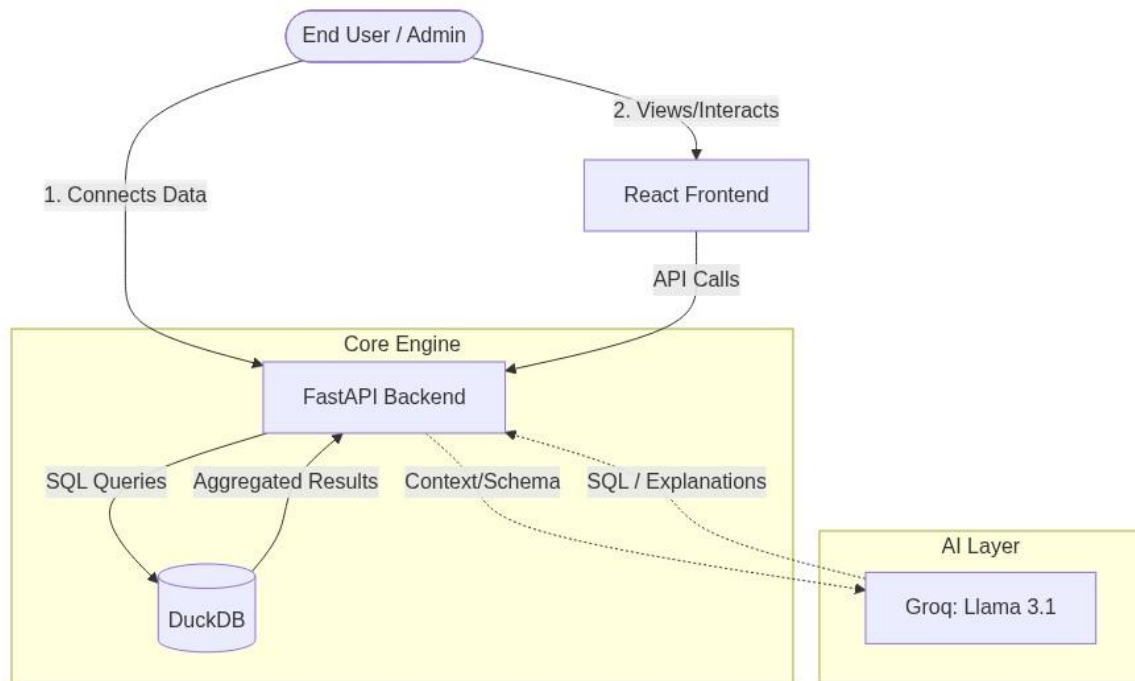
MediQuery currently supports robust data integration pathways to connect to enterprise data:

- **Direct Database Connections (Live):** MediQuery natively supports direct integration with external relational databases, including **MySQL** and **PostgreSQL**. Administrators can securely connect to their existing infrastructure using standard connection strings, allowing MediQuery to instantly ingest, analyze, and visualize live enterprise data without manual exports.
- **Direct File Upload: Administrators** can securely upload structured flat files (CSV, TSV, or Excel). The backend seamlessly parses the file and loads it into the high-performance Duck DB analytical engine for instantaneous querying.

### 4. System Architecture & Workflows

#### System Diagram

The following diagram outlines the overall architecture, distinguishing between the core execution engine and the LLM API layer.



#### How Charts are Generated

1. **Initial Discovery:** When data is uploaded, the backend generates a 'Data Dictionary' containing column names, types, and sample data.

2. **Baseline Generation via AI:** This schema is securely sent to the Groq API (Llama 3.1). The AI responds with a JSON array outlining 4-6 foundational charts, mapping columns to X/Y axes and applying basic aggregations.
3. **Admin Orchestration:** The administrator reviews the baseline, modifies colors, adds Calculated Fields, and manually creates any additional charts using the Drag-and-Drop UI. No AI is used for manual chart creation.
4. **Execution:** The React frontend sends chart configurations to the FastAPI backend, which translates them into blazing-fast DuckDB SQL queries. DuckDB aggregates the data locally and returns only the summary values to the frontend to render the SVGs.

## How "Ask AI" Works

5. **Context Gathering:** When a user asks a question via the Chat interface, the frontend packages the query along with the schema and the metadata of the currently visible dashboard charts.
6. **Querying the LLM:** The prompt is sent to the Groq Llama-3.1 model. The AI is instructed to act as a data analyst. It decides whether to generate a SQL query to answer a specific numerical question or simply provide a natural language explanation of the data.
7. **Execution & Response:** If SQL is generated, the backend executes it against DuckDB and feeds the result back to the LLM to summarize, or the frontend directly displays the answer/chart to the user.

## 5. Technologies Used

- **Frontend:** React, Vite, Material-UI (MUI), Recharts, and React-Grid-Layout.
- **Backend:** FastAPI (Python) and Uvicorn for asynchronous, high-throughput API endpoints.
- **Database Engine:** DuckDB — An embedded analytical SQL database optimized for blazing-fast OLAP queries.
- **AI & NLP Engine:** Groq API utilizing the 'llama-3.1-8b-instant' model for initial schema parsing and chat functionality.

## 6. Deployment Strategy

MediQuery is built with a modern, container-friendly architecture that can be deployed across various cloud providers (AWS, Azure, GCP) or on-premise infrastructure.

- **Dockerization:** Both the frontend and backend can be containerized using Docker. The backend runs as a stateless container, while the frontend is served as static HTML/JS assets via Nginx.  
**Zero Ongoing API Costs:** Because the AI is only used during initial data upload or explicit 'Ask AI' queries, daily end-user dashboard traffic does not trigger LLM API calls. This makes the platform incredibly cost-effective at scale.

## 7. Limits & System Capabilities

By restricting AI usage to the initial data onboarding phase and optional chat interactions, the platform gracefully bypasses typical AI limitations:

- **Llama 3.1 8B Output Limits (8,192 Tokens):** The Llama-3.1-8b-instant model has a maximum output limit of 8,192 tokens. Because MediQuery only asks the AI to generate a few foundational charts initially, it comfortably stays well within this token limit.
- **Unlimited Administrator Dashboarding:** Because manual chart creation relies entirely on the local backend DuckDB instance and uses 0 LLM tokens, Administrators can add an unlimited number of widgets without hitting any Groq token caps or rate limits.
- **Groq Rate Limits Avoided:** Groq imposes Limits on Requests Per Minute (RPM) and Tokens Per Minute (TPM) based on pricing tiers. However, because end-users consuming the dashboard do not generate new charts via AI, millions of users can interact with the dashboard simultaneously without sending a single request to Groq.